Week 4 - AWS Systems Manager (Run Command, State Manager, Patch Manager, Inventory, Session Manager, Parameter Store, Automation, Distributor, Fleet Manager)

DOP-C02 Domain 2 (Configuration Management and IaC), also Domains 4 and 6. Systems Manager (SSM) is one of the **most-tested services**. Typical questions:

1. Run, patch, configure or log in to **hundreds of instances** with **no SSH, no bastion, no open ports**.
2. Instances **don't show up** as managed nodes - why?
3. Store config and secrets, get the **latest AMI ID**.
4. **Auto-remediate** non-compliant resources (Config -> **SSM Automation**).
5. Patch on a schedule with **different baselines** for prod and dev.

Part 1 - Foundations: how SSM reaches a node

A **managed node** (EC2 or on-prem / other cloud) needs **all three**:

#	Requirement	Details
1	**SSM Agent** installed and running	Pre-installed on Amazon Linux 2 / 2023, Windows Server AMIs, Ubuntu (snap), macOS AMIs; keep updated with `AWS-UpdateSSMAgent`
2	**IAM permissions**	Instance profile with `AmazonSSMManagedInstanceCore`, or **Default Host Management Configuration (DHMC)** - SSM permissions for every EC2 instance in the account / Region without an instance profile (needs **IMDSv2**)
3	**Network path** (outbound HTTPS 443)	`ssm`, `ssmmessages` (Session Manager), `ec2messages`; private subnet without NAT = **VPC interface endpoints** for all three + **S3 gateway endpoint** (patches, logs, output) + `logs` / `kms` if used; **no inbound ports**

- "Instance not in Fleet Manager / Run Command targets" = **agent not running**, **instance profile missing**, or **no route to the endpoints**.
- **Hybrid / on-prem** nodes (IDs `mi-`): create a **hybrid activation** (code + ID + IAM service role), install the agent and register. **Advanced-instances tier** for more than 1,000 hybrid nodes per account / Region.
- **Quick Setup** configures DHMC, agent updates, inventory, patch policies, CloudWatch agent across an **Organization**.

SSM documents

Type	Used by	Examples
Command	Run Command, State Manager, maintenance windows	`AWS-RunShellScript`, `AWS-RunPowerShellScript`, `AWS-RunPatchBaseline`, `AWS-ConfigureAWSPackage`, `AWS-UpdateSSMAgent`, `AWS-ApplyAnsiblePlaybooks`, `AWS-RunRemoteScript`, `AWS-GatherSoftwareInventory`
Automation (runbook)	Automation, Config remediation, EventBridge, maintenance windows	`AWS-RestartEC2Instance`, `AWS-UpdateLinuxAmi`, `AWS-CreateImage`, `AWSConfigRemediation-*`, `AWSSupport-*`
Session	Session Manager	`AWS-StartPortForwardingSession`, `AWS-StartSSHSession`
Package / Policy / ChangeCalendar	Distributor / Inventory / Change Calendar	AmazonCloudWatchAgent, AWSPVDriver

Documents are **JSON or YAML**, **versioned** (default version), take **parameters**, can be **shared** with accounts or public; `{{ssm:/param}}` reads Parameter Store values.

Part 2 - Run Command (one-time commands at scale)

Run a Command document on many nodes **on demand**, with **no SSH and no inbound ports**.

Feature	Detail
Targets	Instance IDs, **tags** (`Env=prod`), **resource groups**, all nodes
Rate control	**Concurrency** (number or %, e.g. 10%) + **error threshold** (number or %, stop after N fail)
Output	Console shows the first **2,500 characters**; **full output to S3** and / or **CloudWatch Logs**
Notifications	**SNS** on status changes; **EventBridge** command status events
Triggers	Console / CLI / SDK, **EventBridge rule target**, maintenance window task
Security	IAM controls documents and targets (tag conditions); **CloudTrail** records `SendCommand`
Timeouts	Delivery timeout + execution timeout

```
aws ssm send-command --document-name AWS-RunShellScript \
  --targets Key=tag:Env,Values=prod \
  --parameters commands="yum -y update httpd && systemctl restart httpd" \
  --max-concurrency 10% --max-errors 5% \
  --output-s3-bucket-name my-ssm-output \
  --cloud-watch-output-config CloudWatchOutputEnabled=true
```

"Restart a service on 500 instances, 10% at a time, stop if more than 5% fail, no SSH" = **Run Command + tags + concurrency + error threshold**.

Part 3 - State Manager (desired state)

Associations apply a document **on a schedule** (or once) and **reapply** it, **including on new nodes that match the targets**.

Feature	Detail
Association	Document + targets (tags) + **schedule** (`cron(...)` / `rate(30 minutes)`) + parameters
Uses	Keep the **SSM Agent updated**, **install / configure the CloudWatch agent**, **collect Inventory**, domain join, antivirus / firewall settings, Ansible / Chef / Salt, scheduled **Automation runbooks**
New instances	Launched later **with matching tags get the association automatically**
Compliance	Association status **Compliant / Non-compliant** in SSM Compliance (also Config / Security Hub)
Options	`ApplyOnlyAtCronInterval`, rate control, output to S3

	Run Command	State Manager
When	**Once, on demand**	**Scheduled / continuous**
New instances	Not affected	**Picked up automatically**
Goal	Do something now	**Keep a desired state**
Keyword	Run a script now, ad hoc	**Ensure always**, every new instance

Part 4 - Patch Manager

Patch baselines (what to install)

- **Predefined AWS baselines** per OS (for example `AWS-AmazonLinux2023DefaultPatchBaseline`, `AWS-DefaultPatchBaseline`); typically auto-approve Critical / Important security patches **7 days** after release; **can't be edited**.
- **Custom baselines**:

Setting	Meaning
Approval rules	Product, **classification** (Security, Bugfix...), **severity**, **auto-approve after N days** (7 dev, 14 prod) or **approve until a date**
Approved patches	Explicit list (KB IDs, package names)
Rejected patches	Explicit list; action **allow as dependency** or **block**
Compliance level	Severity reported for a missing patch
Non-security updates	Linux option
Patch sources	Custom Linux repositories
Default baseline	**One default per OS** per account / Region (nodes not in a patch group)

Patch groups (which nodes use which baseline)

- Tag key `Patch Group` (or `PatchGroup`), for example `Patch Group = prod-linux`, registered with a baseline.
- **One baseline per OS per patch group**; a node is in only one patch group.
- Modern alternative: **patch policies** (Quick Setup) patch across **accounts and Regions in an Organization**.

AWS-RunPatchBaseline (how patching runs)

Parameter	Values
Operation	`Scan` (report only) or `Install` (install and report)
RebootOption	`RebootIfNeeded` (default) or `NoReboot`
Other	`AWS-RunPatchBaselineWithHooks` (Linux pre / post hooks), `InstallOverrideList`, `SnapshotId`

- Uses the **native package manager** or Windows Update; nodes need access to repositories (NAT, endpoints or mirrors).
- **Patch now** in the console for on-demand patching.
- **A baseline alone patches nothing**: run AWS-RunPatchBaseline via a maintenance window, association or patch policy.

Maintenance windows (when to patch)

Element	Detail
Schedule	cron / rate, **duration**, **cutoff** (no new tasks N hours before end), time zone, start / end dates
Targets	Tags, instance IDs, resource groups
Tasks	**Run Command**, **Automation**, **Lambda**, **Step Functions**, with **priority**, **concurrency**, **error threshold**
Service role	Optional custom role

Compliance and reporting

- Patch Manager compliance reporting, SSM **Compliance**, **Security Hub**, **AWS Config** rule `ec2-managedinstance-patch-compliance-status-check`.
- **Resource Data Sync** to S3 (Athena / QuickSight).

Classic setup: **dev** baseline (7 days), patch group `dev`, window Saturday 02:00 for 3 hours; **prod** baseline (14 days), patch group `prod`, window Sunday 02:00, Install + RebootIfNeeded, concurrency 10%, errors 5%; daily **Scan** association for compliance.

Part 5 - Inventory

- Collects **applications**, AWS components, **files**, network config, **Windows updates**, instance details, **services**, Windows roles, **registry**, **custom inventory**.
- How: **State Manager association** `AWS-GatherSoftwareInventory` (or Quick Setup).
- Query in the console with filters (which instances have OpenSSL below 3.0?).
- **Across accounts / Regions**: **Resource Data Sync -> one S3 bucket -> Athena / QuickSight** (org-wide supported).
- **AWS Config** records inventory changes (`AWS::SSM::ManagedInstanceInventory`) for history and rules.

"All installed software in 20 accounts, queried with SQL" = **Inventory + Resource Data Sync + Athena**.

Part 6 - Session Manager

Browser or CLI shell (and port forwarding) with **no SSH keys, no bastion, no inbound ports, no public IP**.

Feature	Detail
Access control	IAM `ssm:StartSession` with **tag conditions** (developers only `Env=dev`); terminate / resume only own sessions
Auditing	**CloudTrail** `StartSession`; **session data logging** (commands + output) to **S3** and / or **CloudWatch Logs** with KMS; EventBridge
Encryption	TLS + optional **KMS** session encryption (node role needs `kms:Decrypt`)
Preferences	**Run As** (Linux OS user instead of `ssm-user`, tag `SSMSessionRunAs`), idle timeout, max duration, shell profiles
Port forwarding	`AWS-StartPortForwardingSession` (local to instance port); `AWS-StartPortForwardingSessionToRemoteHost` (through the instance to **private RDS** or another host)
SSH / SCP via SSM	`AWS-StartSSHSession` ProxyCommand: no port 22, but **session logging can't see inside SSH**
Client	Console, or AWS CLI + **Session Manager plugin** (`aws ssm start-session --target i-123`)
Network	`ssmmessages` endpoint

- "Remove bastions, close port 22, audit every command" = **Session Manager + logging (S3 / CloudWatch) + KMS**.
- "Laptop to private RDS" = **port forwarding to remote host**.
- EC2 Instance Connect pushes a temporary SSH key and needs **port 22** (or the EIC Endpoint); Session Manager needs **no port**.

Part 7 - Parameter Store

Hierarchical key-value store, for example `/myapp/prod/db/url`.

Type	Notes
`String`	Plain text
`StringList`	Comma-separated
`SecureString`	**KMS-encrypted** (`aws/ssm` or **your CMK**); read needs `ssm:GetParameter` + `kms:Decrypt` + `--with-decryption`

	Standard	Advanced
Cost / count	**Free**, 10,000	Paid, 100,000
Max size	**4 KB**	**8 KB**
Parameter policies	No	**Expiration, ExpirationNotification, NoChangeNotification** (EventBridge)
Cross-account sharing (RAM)	No	Yes
Change tier	Standard to Advanced: yes	**Advanced to Standard: no** (recreate)

- **Throughput**: enable **higher throughput** (extra cost) or **cache** (Parameters and Secrets Lambda extension) when throttled.
- **Versions and labels**: each change = new version; labels (`prod`) point to versions; `name:3` or `name:label`.
- `GetParametersByPath /myapp/prod --recursive --with-decryption`; IAM by path (`parameter/myapp/prod/*`).
- **Public parameters**: **latest AMI** `/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64` (also Windows, ECS-optimized, Bottlerocket), Regions, services.
- **Secrets Manager reference** `/aws/reference/secretsmanager/<name>`; **EventBridge** events on changes.

Integrations

Where	How
CloudFormation	Parameter type `AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>`; dynamic references `{{resolve:ssm:/path:version}}`, `{{resolve:ssm-secure:/path:version}}` (supported properties only)
ECS	`secrets: [{ name, valueFrom: <parameter ARN> }]`; **task execution role** needs `ssm:GetParameters` + `kms:Decrypt`
CodeBuild	`env: parameter-store: { DB_PASS: /app/db/pass }`
Lambda / SSM docs	SDK or **Parameters and Secrets Lambda extension**; `{{ssm:/path}}` in documents

Parameter Store vs Secrets Manager

Need	Choose
Automatic rotation (RDS / Aurora / Redshift / DocumentDB built in, Lambda for others)	**Secrets Manager**
Cross-account **resource policy**, **multi-Region replication**, **64 KB**	**Secrets Manager**
Config + simple encrypted values, **free**, hierarchy, **public AMI parameters**	**Parameter Store**
Notify when expiring / not changed in N days	Parameter Store **Advanced policies**

Part 8 - Automation (runbooks)

Automation documents perform **multi-step operational tasks on AWS resources** (API calls, scripts, approvals, branching). **The agent is only needed for steps that run commands on instances.**

Action	Use
`aws:executeAwsApi`	Any AWS API
`aws:runCommand`	Command document on instances
`aws:executeScript`	Inline Python / PowerShell
`aws:invokeLambdaFunction`, `aws:executeStateMachine`	Lambda / Step Functions
`aws:approve`	**Manual approval** (SNS, approvers)
`aws:branch`	Conditional logic
`aws:waitForAwsResourceProperty`, `aws:assertAwsResourceProperty`	Wait for / check state
`aws:createImage`, `aws:copyImage`, `aws:runInstances`, `aws:changeInstanceState`, `aws:createStack`, `aws:sleep`	Resource operations

AWS-owned runbooks to recognise:

- `AWS-RestartEC2Instance`, `AWS-StopEC2Instance`, `AWS-CreateImage`
- `AWS-UpdateLinuxAmi` / `AWS-UpdateWindowsAmi` (golden AMI: launch, patch, create AMI, terminate) - EC2 Image Builder is newer
- `AWS-PatchInstanceWithRollback`
- `AWSSupport-ExecuteEC2Rescue`, `AWSSupport-ResetAccess` (lost SSH key / Windows password), `AWSSupport-TroubleshootSSH`
- `AWSConfigRemediation-*` (EnableS3BucketEncryption, RemoveUnrestrictedSourceIngressRules), `AWS-DisablePublicAccessForSecurityGroup`

Execution

- Modes: **simple**, **rate control** (targets, concurrency, error threshold), **manual** (step by step), **multi-account / multi-Region** (management or delegated admin with `AWS-SystemsManager-AutomationAdministrationRole` + `AutomationExecutionRole`).
- Permissions: your identity or `AutomationAssumeRole` (service role) - use it for EventBridge, Config remediation, least privilege.
- Triggers: console / CLI, **EventBridge target**, **AWS Config remediation (manual or automatic)**, **State Manager**, **maintenance window**, **Change Manager**, **OpsCenter**, Incident Manager.
- **Change Calendar** blocks automation during freeze periods.

Classic: "**automatically fix** unencrypted buckets / SGs open to 0.0.0.0/0" = **Config rule + automatic remediation with an SSM Automation runbook** (service role).

Part 9 - Distributor and Fleet Manager

Distributor

- Package software (**zip per platform / architecture +** `manifest.json` with install / uninstall scripts) in S3, published as a **Package document**; **versioned**, **shareable**.
- Install **once** with Run Command `AWS-ConfigureAWSPackage`, or **continuously (new instances too)** with a **State Manager association**.
- AWS packages: `AmazonCloudWatchAgent`, `AWSPVDriver`, `AwsEnaNetworkDriver`, `AWSNVMe`, Inspector / Kinesis agents, partner agents.
- Install types: **uninstall and reinstall** or **in-place update**.
- "Custom agent on all Windows and Linux instances, including future ones, with versions" = **Distributor + State Manager**.

Fleet Manager

- One UI for all managed nodes (EC2, on-prem, edge), **no SSH / RDP ports**.
- Node status, agent version, OS, connection issues; **file system** browse / tail; **performance counters**; **logs**; **Windows registry**; **users and groups**; **processes**.
- **Browser Remote Desktop to Windows** (RDP over Session Manager, **no port 3389**).
- "RDP to private Windows instances without 3389 or a bastion" = **Fleet Manager Remote Desktop**.

Other capabilities

Capability	Purpose
OpsCenter	**OpsItems** from EventBridge, alarms, Config; investigate, run runbooks
Explorer	Dashboard of OpsItems, patch compliance, inventory (multi-account)
Incident Manager	Response plans, escalation, on-call, runbooks
Change Manager	Change requests with **approvals** running runbooks
Change Calendar	Open / closed periods that **block or allow actions**
Compliance	Patch + association + custom compliance
Application Manager	Resources grouped by application
AppConfig	Feature flags and config deployment (next topic)

Part 10 - Choosing the right capability

Requirement	Capability
Script on many instances **now**, no SSH	**Run Command**
Always keep agent / config / software, **new instances too**	**State Manager**
Patch with approval rules, prod vs dev, on schedule	**Patch Manager** (baselines + patch groups + windows, or patch policies)
Tasks only in a defined window	**Maintenance Windows**
Installed software everywhere, SQL	**Inventory + Resource Data Sync + Athena**
Shell without bastion / port 22, full audit	**Session Manager**
Laptop to private RDS	**Port forwarding to remote host**
Config / encrypted values, latest AMI	**Parameter Store**
Secrets with automatic rotation	**Secrets Manager**
Multi-step ops, remediation, golden AMI, approvals	**Automation**
Auto-fix non-compliant resources	**Config + SSM Automation remediation**
Package and roll out custom software	**Distributor** (+ State Manager)
Browser RDP, files, processes, no ports	**Fleet Manager**
Block changes during a freeze	**Change Calendar**

Part 11 - Exam traps

Trap	Truth
Instance not managed	**Agent**, `AmazonSSMManagedInstanceCore` (or DHMC), **network to ssm / ssmmessages / ec2messages**
Private subnet without NAT	**Interface endpoints** for the three + S3 gateway endpoint
Session Manager needs port 22 / public IP	**No**, outbound 443 only
On-prem servers	**Hybrid activation** (`mi-`)
Run Command for every new instance	That's **State Manager**
Run Command output truncated	Console 2,500 chars; **S3 / CloudWatch Logs**
Baseline alone patches instances	**No**: AWS-RunPatchBaseline Install via window / association / patch policy
Two baselines for one patch group and OS	**One per OS per patch group**
Edit the AWS predefined baseline	**Can't**; create custom
Report compliance without installing	**Operation Scan**
Don't reboot prod during patching	**RebootOption NoReboot**
Patch group tag key	`Patch Group` / `PatchGroup`
Org-wide patching	**Patch policies** (Quick Setup)
Inventory across accounts	**Resource Data Sync -> S3 -> Athena**
Audit every session command	**Session logging S3 / CloudWatch** (not inside SSH tunnels)
Limit which instances a user can access	`ssm:StartSession` with **resource tag conditions**
SecureString AccessDenied	Missing `kms:Decrypt` (or `--with-decryption`)
Parameter over 4 KB	**Advanced** (8 KB) or Secrets Manager
Parameter expiry notification	**Advanced policies**
Downgrade Advanced to Standard	**Not possible**
Latest AMI in CloudFormation	`AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>` + public path
Need rotation	**Secrets Manager**
GetParameter throttling	**Higher throughput** or caching
Auto-remediate Config non-compliance	**SSM Automation** (AutomationAssumeRole)
Automation needs the agent	Only for `aws:runCommand` steps
Human approval in a runbook	`aws:approve`
Lost EC2 key pair	`AWSSupport-ResetAccess` / EC2Rescue
Custom agent on current + future instances	**Distributor + State Manager**
Windows RDP without 3389	**Fleet Manager Remote Desktop**
Stop automation in a holiday freeze	**Change Calendar**

Part 12 - Hands-on lab (about 90 minutes, low cost, t3.micro)

Lab 1 - Managed nodes and Fleet Manager

1. Launch two Amazon Linux 2023 instances: **no key pair, no inbound rules**, instance profile **AmazonSSMManagedInstanceCore**, tags `Env=dev` / `Env=prod` and `Patch Group=dev` / `prod`.
2. **Fleet Manager**: both online; browse files and performance counters.
3. Remove the instance profile from one node: **connection lost**. Re-attach.

Lab 2 - Run Command

1. `AWS-RunShellScript`, target `tag:Env=prod`, command `uname -a; df -h`, concurrency 50%, errors 1, output to S3 + CloudWatch Logs.
2. Check output in S3; add an **SNS notification** for failures.
3. Install the CloudWatch agent with `AWS-ConfigureAWSPackage` (Install, `AmazonCloudWatchAgent`) - this is **Distributor**; later make it a State Manager association.

Lab 3 - Session Manager with logging

1. Preferences: **CloudWatch Logs + S3 logging**, **KMS encryption** (node role `kms:Decrypt`), idle timeout.
2. Start a session, run commands, find them in CloudWatch Logs; check **CloudTrail** `StartSession`.
3. From the CLI + plugin: `aws ssm start-session --target i-xxx`, then port forwarding with `--document-name AWS-StartPortForwardingSession --parameters portNumber=80,localPortNumber=8080`.
4. Role allowing `ssm:StartSession` only on `Env=dev`: confirm prod is **denied**.

Lab 4 - Parameter Store

```
aws ssm put-parameter --name /lab/dev/db/url --type String --value "db.dev.local"
aws ssm put-parameter --name /lab/dev/db/pass --type SecureString --value "S3cret!"
aws ssm get-parameters-by-path --path /lab/dev --recursive --with-decryption
aws ssm get-parameter --name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64
```

Update `/lab/dev/db/url` twice, run `get-parameter-history`, then `label-parameter-version --labels prod`.

Lab 5 - State Manager and Inventory

1. Quick Setup Host Management, or associations `AWS-GatherSoftwareInventory` (30 minutes) and `AWS-UpdateSSMAgent` (weekly).
2. **Inventory**: filter applications (`openssl`); optional **Resource Data Sync** to S3 + Athena.
3. Launch a third `Env=dev` instance: associations **apply automatically**.

Lab 6 - Patch Manager

1. Custom baseline (Amazon Linux 2023, Security, Critical / Important, 7 days); register patch group `dev`.
2. `AWS-RunPatchBaseline` **Operation=Scan** on all nodes; view **compliance**.
3. Maintenance window `cron(0 2 ? * SUN *)`, duration 3h, cutoff 1h, target `Patch Group=dev`, task AWS-RunPatchBaseline Install + RebootIfNeeded, concurrency 50%, errors 1.

Lab 7 - Automation and Config remediation

1. Run `AWS-StopEC2Instance`, then `AWS-StartEC2Instance`.
2. Custom YAML runbook: `aws:approve` (SNS), then `aws:executeAwsApi` (CreateTags); run and approve.
3. Optional: Config rule `s3-bucket-server-side-encryption-enabled` + **auto-remediation** `AWSConfigRemediation-EnableS3BucketEncryption`.

Cleanup: terminate instances; delete the window, associations, baseline, parameters, data sync, Config rules, logs and buckets.

Part 13 - Check yourself

Questions

1. What three things does an EC2 instance need to become a managed node?
2. Private-subnet instances without NAT don't show in Fleet Manager. What do you add?
3. Run Command or State Manager: "CloudWatch agent on all current and future web servers"?
4. Run a script on 1,000 instances, 5% at a time, stop after 10 failures, keep full output - how?
5. Patch baseline vs patch group vs maintenance window?
6. Approve security patches after 7 days in dev and 14 days in prod - how?
7. What do `Operation=Scan` and `RebootOption=NoReboot` do?
8. All installed software across 30 accounts, queried with SQL - how?
9. Replace bastions and audit every admin command - how?
10. Laptop to private RDS with no public endpoint and no bastion SSH - how?
11. Standard vs Advanced parameters? Extra permission for SecureString?
12. How does a CloudFormation template always use the latest Amazon Linux AMI?
13. DB password must rotate every 30 days: Parameter Store or Secrets Manager?
14. Automatically remediate security groups open to 0.0.0.0/0 on port 22 - how?
15. Which capabilities: custom agent everywhere, browser RDP without 3389, block automation in a holiday freeze?

Answers

1. **SSM Agent**, **IAM** (instance profile with AmazonSSMManagedInstanceCore or DHMC), **outbound 443** to ssm / ssmmessages / ec2messages.
2. **Interface endpoints** for ssm, ssmmessages, ec2messages (+ S3 gateway endpoint).
3. **State Manager** (reapplies, picks up new tagged instances).
4. **Run Command**, tag targets, **max concurrency 5%**, **max errors 10**, output to **S3 / CloudWatch Logs**.
5. **Baseline** = which patches (rules); **patch group** = `Patch Group` tag mapping nodes to a baseline; **maintenance window** = when (schedule + tasks).
6. **Two custom baselines** (7 and 14 days) registered to **dev** and **prod** patch groups.
7. **Scan** reports missing patches only; **NoReboot** installs without rebooting.
8. **Inventory** + **Resource Data Sync** to one S3 bucket + **Athena** (QuickSight).
9. **Session Manager** with **logging to S3 / CloudWatch (KMS)**, CloudTrail, IAM tag conditions.
10. `AWS-StartPortForwardingSessionToRemoteHost` through a managed instance.
11. Standard free 4 KB no policies; **Advanced 8 KB, policies, RAM sharing, no downgrade**; SecureString needs `kms:Decrypt`.
12. Parameter type `AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>` with the public AMI path as default.
13. **Secrets Manager**.
14. **AWS Config rule + automatic remediation** with an **SSM Automation runbook** and AutomationAssumeRole.
15. **Distributor (+ State Manager)**, **Fleet Manager Remote Desktop**, **Change Calendar**.

Quick memory summary

- **Managed node** = agent + AmazonSSMManagedInstanceCore / DHMC + 443 to ssm / ssmmessages / ec2messages (VPC endpoints); **no inbound ports**; hybrid = activation (`mi-`).
- **Run Command** = once, now (rate control, output S3 / CW); **State Manager** = desired state on a schedule, **new instances included**.
- **Patch Manager**: **baseline** + **patch group** (one baseline per OS) + **maintenance window** (AWS-RunPatchBaseline Scan / Install) or **patch policies**.
- **Inventory -> Resource Data Sync -> S3 / Athena**.
- **Session Manager**: no SSH / bastion, IAM tags, **logging S3 / CW + KMS**, port forwarding (remote host).
- **Parameter Store**: String / StringList / **SecureString**, Standard 4 KB vs **Advanced 8 KB + policies**, hierarchy, labels, **public AMI params**, CFN integration; **rotation = Secrets Manager**.
- **Automation**: runbooks (API, scripts, **approve**, branch) from **Config remediation**, EventBridge, windows; golden AMI, EC2Rescue, ResetAccess.
- **Distributor** = package + install; **Fleet Manager** = UI + **browser RDP**; **Change Calendar** = freezes.